

```

1 import pandas as pd
2
3 df_csv = pd.read_csv('maharashtra_weather_5year.csv')
4 display(df_csv.head())

```

	year	month	district	temperature	humidity	rainfall_mm	wind_speed	condition
0	2020	1	Ahmednagar	24	44	15	5	Cloudy
1	2020	1	Akola	36	42	17	9	Cloudy
2	2020	1	Amravati	23	66	0	4	Cloudy
3	2020	1	Aurangabad	37	76	18	8	Cloudy
4	2020	1	Beed	26	81	20	3	Cloudy

Step 1: Redefine Target Variables and Split Data

Since we now want to predict `temperature`, `wind_speed`, and `condition`, I'll define these as our new target variables and split the data into training and testing sets accordingly. The features (X) will remain `year`, `month`, and `district_encoded`.

```

1 from sklearn.model_selection import train_test_split
2
3 # Define features (X)
4 X = df_encoded[['year', 'month', 'district_encoded']]
5
6 # Define all target variables
7 y_targets = df_encoded[['temperature', 'wind_speed', 'condition_encoded']]
8
9 # Split the data into training and testing sets (80% train, 20% test)
10 # We'll stratify by 'condition_encoded' to maintain class distribution in splits.
11 X_train, X_test, y_targets_train, y_targets_test = train_test_split(
12     X, y_targets, test_size=0.2, random_state=42, stratify=df_encoded['condition_encoded']
13 )
14
15 # Separate the targets for individual models
16 y_temp_train = y_targets_train['temperature']
17 y_temp_test = y_targets_test['temperature']
18
19 y_wind_train = y_targets_train['wind_speed']
20 y_wind_test = y_targets_test['wind_speed']
21
22 y_condition_train = y_targets_train['condition_encoded']
23 y_condition_test = y_targets_test['condition_encoded']
24
25 print(f"Training set size: {len(X_train)} samples")
26 print(f"Testing set size: {len(X_test)} samples")

```

Training set size: 1728 samples
Testing set size: 432 samples

Step 2: Train and Evaluate Multiple Models

Now, I will train a `RandomForestRegressor` for temperature, another `RandomForestRegressor` for wind speed, and a `RandomForestClassifier` for the weather condition. After training, I'll evaluate their performance using Mean Absolute Error (MAE) for regression and Accuracy for classification.

```
1 from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
2 from sklearn.metrics import mean_absolute_error, accuracy_score
3
4 # --- Temperature Model (Regression) ---
5 temp_model = RandomForestRegressor(random_state=42)
6 temp_model.fit(X_train, y_temp_train)
7 y_temp_pred = temp_model.predict(X_test)
8 mae_temp = mean_absolute_error(y_temp_test, y_temp_pred)
9 print(f"Temperature Model MAE: {mae_temp:.4f}")
10
11 # --- Wind Speed Model (Regression) ---
12 wind_model = RandomForestRegressor(random_state=42)
13 wind_model.fit(X_train, y_wind_train)
14 y_wind_pred = wind_model.predict(X_test)
15 mae_wind = mean_absolute_error(y_wind_test, y_wind_pred)
16 print(f"Wind Speed Model MAE: {mae_wind:.4f}")
17
18 # --- Condition Model (Classification) ---
19 condition_model = RandomForestClassifier(random_state=42)
20 condition_model.fit(X_train, y_condition_train)
21 y_condition_pred = condition_model.predict(X_test)
22 accuracy_condition = accuracy_score(y_condition_test, y_condition_pred)
23 print(f"Condition Model Accuracy: {accuracy_condition:.4f}")
```

Temperature Model MAE: 4.7943
Wind Speed Model MAE: 2.8266
Condition Model Accuracy: 1.0000

▼ Step 3: Test Models with New Input

Now you can enter a year, month, and district to get predictions for temperature, wind speed, and weather condition.

```
1 import pandas as pd
2
3 # Take user input
4 input_year = int(input("Enter the year (e.g., 2023): "))
5 input_month = int(input("Enter the month (1-12): "))
6 input_district = input("Enter the district (e.g., Nashik, Pune): ").title() # Convert to title case
7
8 # Encode the district input. Handle cases where district might not be in the training data.
9 try:
10     input_district_encoded = le_district.transform([input_district])[0]
11 except ValueError:
12     print(f"District '{input_district}' not found in the training data. Please choose from: {' '.join(le_district.classes_}")
13     input_district_encoded = None
14
15 if input_district_encoded is not None:
16     # Create a DataFrame for the single input
17     new_data = pd.DataFrame([[input_year, input_month, input_district_encoded]],
18                             columns=['year', 'month', 'district_encoded'])
19
20 # Make predictions with each model
21 predicted_temperature = temp_model.predict(new_data)[0]
22 predicted_wind_speed = wind_model.predict(new_data)[0]
```

```

23 predicted_condition_encoded = condition_model.predict(new_data)[0]
24
25 # Decode the predicted condition
26 predicted_condition = le_condition.inverse_transform([predicted_condition_encoded])[0]
27
28 print(f"\nPredictions for {input_district} in {input_month}/{input_year}:")
29 print(f" Predicted Temperature: {predicted_temperature:.2f} °C")
30 print(f" Predicted Wind Speed: {predicted_wind_speed:.2f} km/h")
31 print(f" Predicted Condition: {predicted_condition}")

```

Enter the year (e.g., 2023): 2026
Enter the month (1-12): 3
Enter the district (e.g., Nashik, Pune): Thane

Predictions for Thane in 3/2026:
Predicted Temperature: 23.97 °C
Predicted Wind Speed: 10.58 km/h
Predicted Condition: Hot

2nd Model

▼ Step 1: Redefine Features and Target for District Prediction

Now, I'll set up the data for predicting `district`. The input features (`X`) will be `year`, `month`, `temperature`, `condition_encoded`, `rainfall_mm`, and `humidity`. The target (`y`) will be `district_encoded`.

```

1 from sklearn.model_selection import train_test_split
2
3 # Define new features (X) and target (y) for district prediction
4 X_district = df_encoded[['year', 'month', 'temperature', 'condition_encoded', 'rainfall_mm', 'humidity']]
5 y_district = df_encoded['district_encoded']
6
7 # Split the data into training and testing sets (80% train, 20% test)
8 X_district_train, X_district_test, y_district_train, y_district_test = train_test_split(
9     X_district, y_district, test_size=0.2, random_state=42, stratify=y_district
10 )
11
12 print(f"Training set size for district model: {len(X_district_train)} samples")
13 print(f"Testing set size for district model: {len(X_district_test)} samples")

```

Training set size for district model: 1728 samples
Testing set size for district model: 432 samples

▼ Step 2: Train and Evaluate District Prediction Model

I will now train a `RandomForestClassifier` to predict the district and evaluate its performance using accuracy.

```

1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.metrics import accuracy_score
3
4 # Initialize the RandomForestClassifier model for district prediction
5 district_model = RandomForestClassifier(random_state=42)
6
7 # Train the model using the training data
8 district_model.fit(X_district_train, y_district_train)

```

```
9
10 # Make predictions on the test set
11 y_district_pred = district_model.predict(X_district_test)
12
13 # Calculate the accuracy of the model
14 accuracy_district = accuracy_score(y_district_test, y_district_pred)
```

District Model Accuracy: 0.0162

▼ Step 3: Test District Prediction Model with New Input

Now you can enter a year, month, temperature, condition, rainfall, and humidity to get a predicted district.

```
1 import pandas as pd
2
3 # Take user input for new features
4 input_year_dist = int(input("Enter the year (e.g., 2023): "))
5 input_month_dist = int(input("Enter the month (1-12): "))
6 input_temperature_dist = float(input("Enter the temperature (°C): "))
7 input_condition_dist = input("Enter the condition (Hot, Cloudy, Rain): ").title() # Convert to title case
8 input_rainfall_dist = float(input("Enter the rainfall (mm): "))
9 input_humidity_dist = int(input("Enter the humidity (%): "))
10
11 # Encode the input condition
12 try:
13     input_condition_encoded_dist = le_condition.transform([input_condition_dist])[0]
14 except ValueError:
15     print(f"Condition '{input_condition_dist}' not found in the training data. Please choose from: {'', '.join(le_condition.classes_)}")
16     input_condition_encoded_dist = None
17
18 if input_condition_encoded_dist is not None:
19     # Create a DataFrame for the single input
20     new_data_district = pd.DataFrame([[
21         input_year_dist, input_month_dist, input_temperature_dist,
22         input_condition_encoded_dist, input_rainfall_dist, input_humidity_dist
23     ]],
24         columns=['year', 'month', 'temperature', 'condition_encoded', 'rainfall_mm', 'humidity'])
25
26     # Make prediction with the district model
27     predicted_district_encoded = district_model.predict(new_data_district)[0]
28
29     # Decode the predicted district
30     predicted_district = le_district.inverse_transform([predicted_district_encoded])[0]
31
32     print(f"\nPredicted District for the given weather conditions: {predicted_district}")
```

```
Enter the year (e.g., 2023): 2024
Enter the month (1-12): 8
Enter the temperature (°C): 25
Enter the condition (Hot, Cloudy, Rain): Hot
Enter the rainfall (mm): 0
Enter the humidity (%): 81
```

Predicted District for the given weather conditions: Nashik

1 Start coding or [generate](#) with AI.

